

Threat Modeling Report

Created on 6/3/2026 12:45:14 AM

Threat Model Name: Threat Model for Aaronic Oil & Energy Website

Owner: Anene Ifunanya

Reviewer: Timmie'O

Contributors: Cybersecurity Internship Team (8 members)

Description: A comprehensive threat model analysis for Aaronic Oil & Energy e-commerce platform using STRIDE methodology. This analysis identifies potential security vulnerabilities across frontend, backend, and database components with 48 identified threats and comprehensive mitigation strategies.

Assumptions: Web application deployed on cloud infrastructure - All data transmitted using HTTPS/TLS encryption - Backend uses SQL database with access control mechanisms - System includes continuous security monitoring - Users access via standard web browsers - Payment processing delegated to third-party provider

External Dependencies: Cloud Infrastructure Provider (AWS, Azure, or GCP) - Third-Party Payment Processing Service - DDoS Protection Service (Cloudflare) - Email/Notification Service - Authentication Service (OAuth 2.0)

Threat Model Summary:

Not Started	0
Not Applicable	0
Needs Investigation	0
Mitigation Implemented	48
Total	48
Total Migrated	0

Diagram: Diagram 1

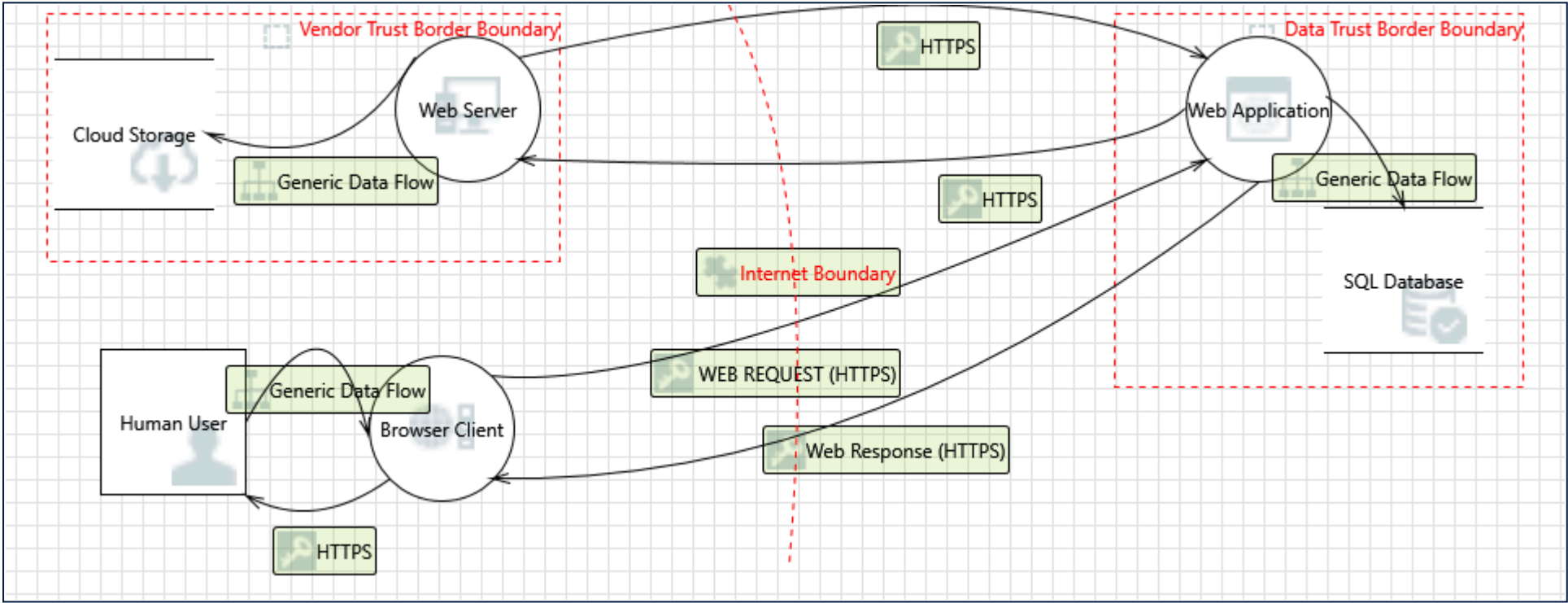
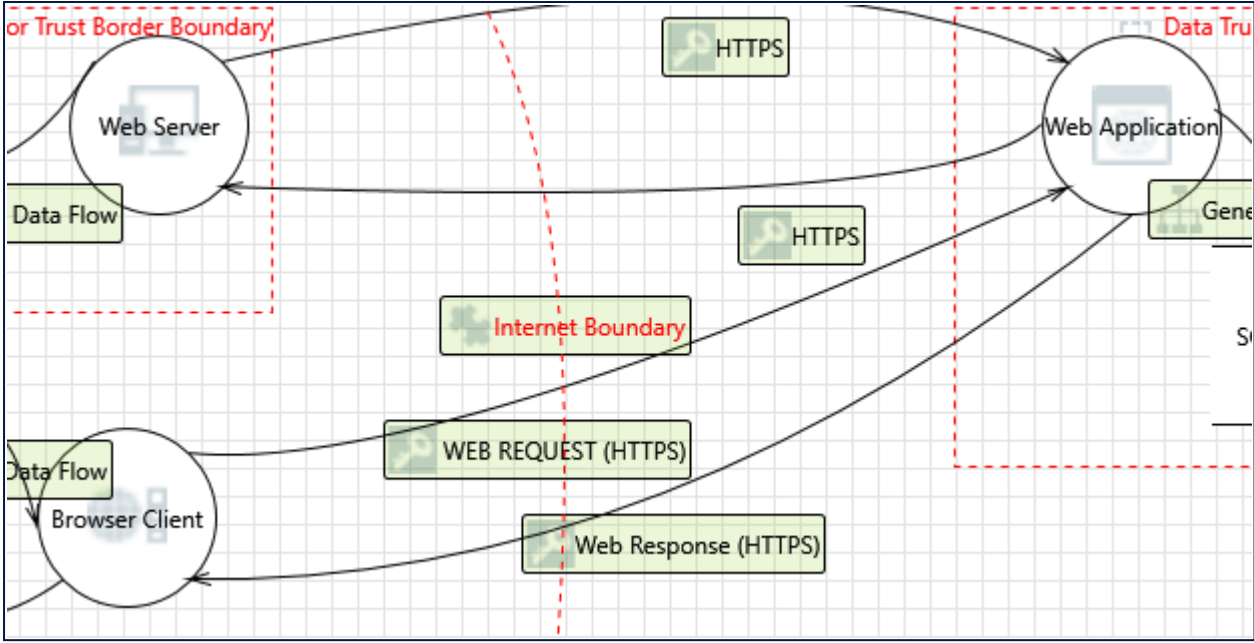


Diagram 1 Diagram Summary:

Not Started	0
Not Applicable	0
Needs Investigation	0
Mitigation Implemented	48
Total	48
Total Migrated	0

Interaction: WEB REQUEST (HTTPS)



1. Replay Attacks [State: Mitigation Implemented] [Priority: Low]

Category: Tampering

Description: Packets or messages without sequence numbers or timestamps can be captured and replayed in a wide variety of ways. Implement or utilize an existing communication protocol that supports anti-replay techniques (investigate sequence numbers before timers) and strong integrity.

Justification: Include a timestamp in each message and reject messages that are too old. Sign messages using HMAC (Hash-based Message Authentication Code) with a shared secret key.

2. Browser Client Process Memory Tampered [State: Mitigation Implemented] [Priority: Low]

Category: Tampering

Description: If Browser Client is given access to memory, such as shared memory or pointers, or is given the ability to control what Web Application executes (for example, passing back a function pointer.), then Browser Client can tamper with Web Application. Consider if the function could work with less access to memory, such as passing data rather than pointers. Copy in data provided, and then validate it.

Justification: Use APIs instead of shared memory. Use Input Validation & Sanitization. Restrict browser-executed code by using a sandbox

3. Collision Attacks [State: Mitigation Implemented] [Priority: Low]

Category: Tampering

Description: Attackers who can send a series of packets or messages may be able to overlap data. For example, packet 1 may be 100 bytes starting at offset 0. Packet 2 may be 100 bytes starting at offset 25. Packet 2 will overwrite 75 bytes of packet 1. Ensure you reassemble data before filtering it, and ensure you explicitly handle these sorts of cases.

Justification: Drop overlapping fragments, Reject packets that overwrite previous data. Detect and block malicious packets. Prevent unnecessary fragmentation. Validate message integrity.

4. Potential Data Repudiation by Web Application [State: Mitigation Implemented] [Priority: Low]

Category: Repudiation

Description: Web Application claims that it did not receive data from a source outside the trust boundary. Consider using logging or auditing to record the source, time, and summary of the received data.

Justification: Log and Audit Data Transfers. Implement strong authentication and authorization mechanisms.

5. Weak Authentication Scheme [State: Mitigation Implemented] [Priority: Low]

Category: Information Disclosure

Description: Custom authentication schemes are susceptible to common weaknesses such as weak credential change management, credential equivalence, easily guessable credentials, null credentials, downgrade authentication or a weak credential change management system. Consider the impact and potential mitigations for your custom authentication scheme.

Justification: Ensure 2FA before credential changes, Enforce strong password policies, Disallow blank passwords and prompt users to set a secure one, Use strong hashing algorithms for storing credentials.

6. Potential Process Crash or Stop for Web Application [State: Mitigation Implemented] [Priority: Low]

Category: Denial Of Service

Description: Web Application crashes, halts, stops or runs slowly; in all cases violating an availability metric.

Justification: We implemented a back up system to create room for redundancy. Cloudfare can also be implemented for ddos protection

7. Data Flow WEB REQUEST (HTTPS) Is Potentially Interrupted [State: Mitigation Implemented] [Priority: Low]

Category: Denial Of Service

Description: An external agent interrupts data flowing across a trust boundary in either direction.

Justification: Authentication and Authorization processes should be put in place. We implemented a back up system to create room for redundancy. Cloudfare can also be implemented for ddos protection

8. Elevation Using Impersonation [State: Mitigation Implemented] [Priority: Low]

Category: Elevation Of Privilege

Description: Web Application may be able to impersonate the context of Browser Client in order to gain additional privilege.

Justification: We implemented an authentication mechanism by using tokens signed by the web application.

9. Web Application May be Subject to Elevation of Privilege Using Remote Code Execution [State: Mitigation Implemented] [Priority: Low]

Category: Elevation Of Privilege

Description: Browser Client may be able to remotely execute code for Web Application.

Justification: Use input validation and parameterized queries. Implement CSP to block unauthorized scripts.Enforce Authentication & Authorization

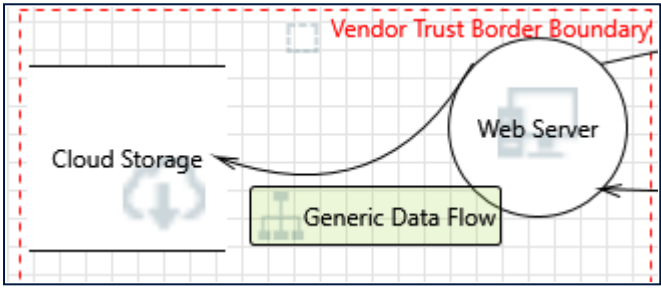
10. Elevation by Changing the Execution Flow in Web Application [State: Mitigation Implemented] [Priority: Low]

Category: Elevation Of Privilege

Description: An attacker may pass data into Web Application in order to change the flow of program execution within Web Application to the attacker's choosing.

Justification: ensure input validation. Use parameterized queries and avoid direct input execution. Enforce Authentication & Authorization

Interaction: Generic Data Flow



11. Risks from Logging [State: Mitigation Implemented] [Priority: Low]

Category: Tampering

Description: Log readers can come under attack via log files. Consider ways to canonicalize data in all logs. Implement a single reader for the logs, if possible, in order to reduce attack surface area. Be sure to understand and document log file elements which come from untrusted sources.

Justification: Avoid evaluating log data as shell commands or code. Ensure log readers cannot modify logs (only view or export them).Store logs in append-only storage to prevent modification

12. Spoofing of Destination Data Store Cloud Storage [State: Mitigation Implemented] [Priority: Low]

Category: Spoofing

Description: Cloud Storage may be spoofed by an attacker and this may lead to data being written to the attacker's target instead of Cloud Storage. Consider using a standard authentication mechanism to identify the destination data store.

Justification: Enforce Strong Authentication & Authorization. Encrypt Data Before Transmission & At Rest. Use mutual TLS (mTLS) to ensure both client and cloud storage authenticate each other.

13. Lower Trusted Subject Updates Logs [State: Mitigation Implemented] [Priority: Low]

Category: Repudiation

Description: If you have trust levels, is anyone other outside of the highest trust level allowed to log? Letting everyone write to your logs can lead to repudiation problems. Only allow trusted code to log.

Justification: Restrict Logging Access by Trust Level. Prevent lower-trust users from modifying or deleting logs. Require MFA before high-trust users can view or modify logs. Separate Log Storage for Different Trust Levels

14. Data Logs from an Unknown Source [State: Mitigation Implemented] [Priority: Low]

Category: Repudiation

Description: Do you accept logs from unknown or weakly authenticated users or systems? Identify and authenticate the source of the logs before accepting them.

Justification: Use mutual TLS (mTLS) or API keys for authentication. Require logs to be digitally signed. Enforce Strict Input Validation. Encrypt Log Data in Transit & At Rest

15. Insufficient Auditing [State: Mitigation Implemented] [Priority: Low]

Category: Repudiation

Description: Does the log capture enough data to understand what happened in the past? Do your logs capture enough data to understand an incident after the fact? Is such capture lightweight enough to be left on all the time? Do you have enough data to deal with repudiation claims? Make sure you log sufficient and appropriate data to handle a repudiation claims. You might want to talk to an audit expert as well as a privacy expert about your choice of data.

Justification: Logs should contain essential details to reconstruct events accurately (Timestamps, Event type, User ID, Resource accessed, Action performed, Success/Failure status, System Context, Correlation ID). Logs should support forensic analysis and security investigations. Logs should prevent users from denying their actions (Digitally sign logs, Correlate logs across multiple sources, Use immutable storage, record MFA events, Capture session metadata).

16. Potential Weak Protections for Audit Data [State: Mitigation Implemented] [Priority: Low]

Category: Repudiation

Description: Consider what happens when the audit mechanism comes under attack, including attempts to destroy the logs, or attack log analysis programs. Ensure access to the log is through a reference monitor, which controls read and write separately. Document what filters, if any, readers can rely on, or writers should expect

Justification: Store logs in an immutable, append-only format (WORM: Write Once, Read Many). Restrict log access with the principle of least privilege (PoLP). Monitor log retention settings (prevent attackers from reducing log history).

17. Authorization Bypass [State: Mitigation Implemented] [Priority: Low]

Category: Information Disclosure

Description: Can you access Cloud Storage and bypass the permissions for the object? For example by editing the files directly with a hex editor, or reaching it via filesharing? Ensure that your program is the only one that can access the data, and that all other subjects have to use your interface.

Justification: Configure Identity and Access Management (IAM) policies to restrict access to only your application. Ensure all users access storage only through your application API and never directly.

18. Weak Credential Storage [State: Mitigation Implemented] [Priority: Low]

Category: Information Disclosure

Description: Credentials held at the server are often disclosed or tampered with and credentials stored on the client are often stolen. For server side, consider storing a salted hash of the credentials instead of storing the credentials themselves. If this is not possible due to business requirements, be sure to encrypt the credentials before storage, using an SDL-approved mechanism. For client side, if storing credentials is required, encrypt them and protect the data store in which they're stored

Justification: Generate a unique, random salt for each user. Use bcrypt, Argon2, or PBKDF2 for password hashing. Enforce Multi-Factor Authentication (MFA) to reduce the risk of credential theft.

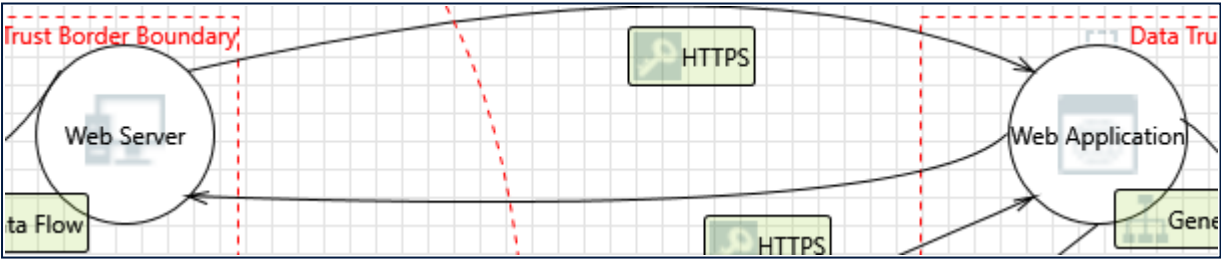
19. Potential Excessive Resource Consumption for Web Server or Cloud Storage [State: Mitigation Implemented] [Priority: Low]

Category: Denial Of Service

Description: Does Web Server or Cloud Storage take explicit steps to control resource consumption? Resource consumption attacks can be hard to deal with, and there are times that it makes sense to let the OS do the job. Be careful that your resource requests don't deadlock, and that they do timeout.

Justification: Implement API Gateway (e.g., Nginx, AWS API Gateway, Cloudflare) with rate-limiting.

Interaction: HTTPS



20. Web Server Process Memory Tampered [State: Mitigation Implemented] [Priority: Low]

Category: Tampering

Description: If Web Server is given access to memory, such as shared memory or pointers, or is given the ability to control what Web Application executes (for example, passing back a function pointer.), then Web Server can tamper with Web Application. Consider if the function could work with less access to memory, such as passing data rather than pointers. Copy in data provided, and then validate it.

Justification: Use APIs instead of shared memory. Use Input Validation & Sanitization. Restrict server-executed code by using a sandbox

21. Weak Authentication Scheme [State: Mitigation Implemented] [Priority: Low]

Category: Information Disclosure

Description: Custom authentication schemes are susceptible to common weaknesses such as weak credential change management, credential equivalence, easily guessable credentials, null credentials, downgrade authentication or a weak credential change management system. Consider the impact and potential mitigations for your custom authentication scheme.

Justification: Ensure 2FA before credential changes, Enforce strong password policies, Disallow blank passwords and prompt users to set a secure one, Use strong hashing algorithms for storing credentials.

22. Elevation Using Impersonation [State: Mitigation Implemented] [Priority: Low]

Category: Elevation Of Privilege

Description: Web Application may be able to impersonate the context of Web Server in order to gain additional privilege.

Justification: We implemented an authentication mechanism by using tokens signed by the server

23. Replay Attacks [State: Mitigation Implemented] [Priority: Low]

Category: Tampering

Description: Packets or messages without sequence numbers or timestamps can be captured and replayed in a wide variety of ways. Implement or utilize an existing communication protocol that supports anti-replay techniques (investigate sequence numbers before timers) and strong integrity.

Justification: Include a timestamp in each message and reject messages that are too old. Sign messages using HMAC (Hash-based Message Authentication Code) with a shared secret key.

24. Potential Data Repudiation by Web Application [State: Mitigation Implemented] [Priority: Low]

Category: Repudiation

Description: Web Application claims that it did not receive data from a source outside the trust boundary. Consider using logging or auditing to record the source, time, and summary of the received data.

Justification: Log and Audit Data Transfers. Implement strong authentication and authorization mechanisms.

25. Potential Process Crash or Stop for Web Application [State: Mitigation Implemented] [Priority: Low]

Category: Denial Of Service

Description: Web Application crashes, halts, stops or runs slowly; in all cases violating an availability metric.

Justification: We implemented a back up system to create room for redundancy. Cloudfare can also be implemented for ddos protection

26. Data Flow HTTPS Is Potentially Interrupted [State: Mitigation Implemented] [Priority: Low]

Category: Denial Of Service

Description: An external agent interrupts data flowing across a trust boundary in either direction.

Justification: We implemented validation of anti forgery token. We also ensured authentication, confidentiality and integrity of data.

27. Web Application May be Subject to Elevation of Privilege Using Remote Code Execution [State: Mitigation Implemented] [Priority: Low]

Category: Elevation Of Privilege

Description: Web Server may be able to remotely execute code for Web Application.

Justification: We sanitized our input data and used sandbox to isolate the web application

28. Elevation by Changing the Execution Flow in Web Application [State: Mitigation Implemented] [Priority: Low]

Category: Elevation Of Privilege

Description: An attacker may pass data into Web Application in order to change the flow of program execution within Web Application to the attacker's choosing.

Justification: The input and output data were sanitized, a sandbox is used to isolate the web application environment and the principle of least privilege is also used.

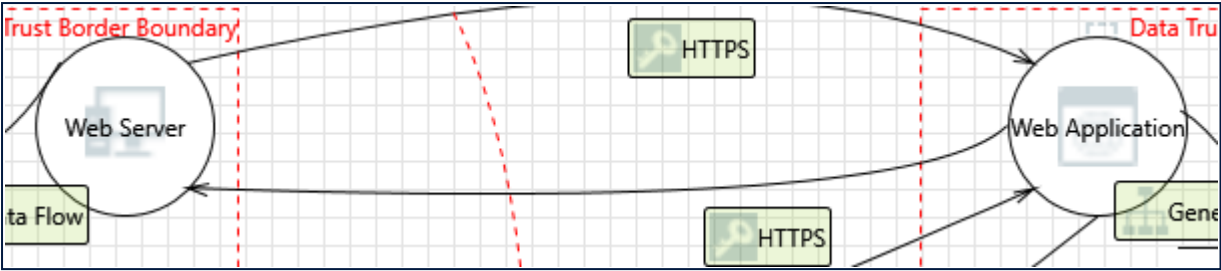
29. Collision Attacks [State: Mitigation Implemented] [Priority: Low]

Category: Tampering

Description: Attackers who can send a series of packets or messages may be able to overlap data. For example, packet 1 may be 100 bytes starting at offset 0. Packet 2 may be 100 bytes starting at offset 25. Packet 2 will overwrite 75 bytes of packet 1. Ensure you reassemble data before filtering it, and ensure you explicitly handle these sorts of cases.

Justification: Drop overlapping fragments, Reject packets that overwrite previous data. Detect and block malicious packets. Prevent unnecessary fragmentation. Validate message integrity.

Interaction: HTTPS



30. Elevation Using Impersonation [State: Mitigation Implemented] [Priority: Low]

Category: Elevation Of Privilege
Description: Web Server may be able to impersonate the context of Web Application in order to gain additional privilege.
Justification: We implemented an authentication mechanism by using tokens signed by the web application

31. Collision Attacks [State: Mitigation Implemented] [Priority: Low]

Category: Tampering
Description: Attackers who can send a series of packets or messages may be able to overlap data. For example, packet 1 may be 100 bytes starting at offset 0. Packet 2 may be 100 bytes starting at offset 25. Packet 2 will overwrite 75 bytes of packet 1. Ensure you reassemble data before filtering it, and ensure you explicitly handle these sorts of cases.
Justification: Drop overlapping fragments, Reject packets that overwrite previous data. Detect and block malicious packets. Prevent unnecessary fragmentation. Validate message integrity.

32. Potential Data Repudiation by Web Server [State: Mitigation Implemented] [Priority: Low]

Category: Repudiation
Description: Web Server claims that it did not receive data from a source outside the trust boundary. Consider using logging or auditing to record the source, time, and summary of the received data.
Justification: Log and Audit Data Transfers. Implement strong authentication and authorization mechanisms.

33. Potential Process Crash or Stop for Web Server [State: Mitigation Implemented] [Priority: Low]

Category: Denial Of Service
Description: Web Server crashes, halts, stops or runs slowly; in all cases violating an availability metric.
Justification: We implemented a back up system to create room for redundancy. Cloudfare can also be implemented for ddos protection

34. Data Flow HTTPS Is Potentially Interrupted [State: Mitigation Implemented] [Priority: Low]

Category: Denial Of Service
Description: An external agent interrupts data flowing across a trust boundary in either direction.
Justification: We implemented a back up system to create room for redundancy. Cloudfare can also be implemented for ddos protection

35. Web Server May be Subject to Elevation of Privilege Using Remote Code Execution [State: Mitigation Implemented] [Priority: Low]

Category: Elevation Of Privilege
Description: Web Application may be able to remotely execute code for Web Server.
Justification: We implemented an authentication mechanism by using tokens signed by the server

36. Elevation by Changing the Execution Flow in Web Server [State: Mitigation Implemented] [Priority: Low]

Category: Elevation Of Privilege
Description: An attacker may pass data into Web Server in order to change the flow of program execution within Web Server to the attacker's choosing.
Justification: Input data is sanitized, authentication and authorization principles are put in place and a sandbox is used to isolate the web server

37. Replay Attacks [State: Mitigation Implemented] [Priority: Low]

Category: Tampering
Description: Packets or messages without sequence numbers or timestamps can be captured and replayed in a wide variety of ways. Implement or utilize an existing communication protocol that supports anti-replay techniques (investigate sequence numbers before timers) and strong integrity.
Justification: Include a timestamp in each message and reject messages that are too old. Sign messages using HMAC (Hash-based Message Authentication Code) with a shared secret key.

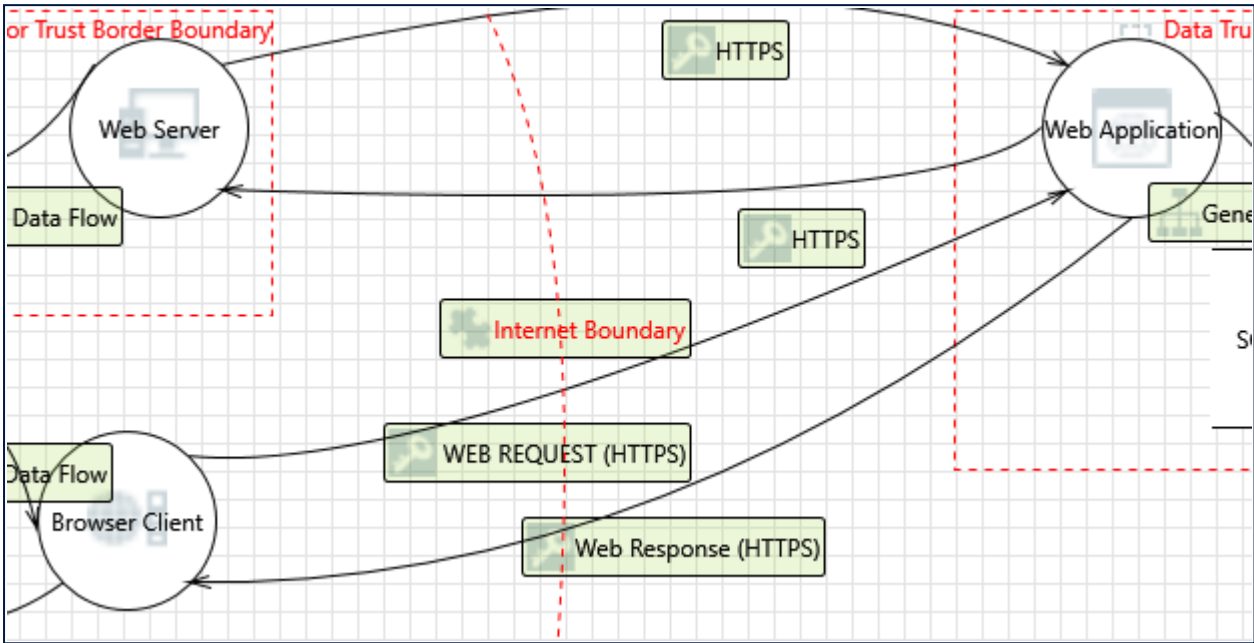
38. Weak Authentication Scheme [State: Mitigation Implemented] [Priority: Low]

Category: Information Disclosure

Description: Custom authentication schemes are susceptible to common weaknesses such as weak credential change management, credential equivalence, easily guessable credentials, null credentials, downgrade authentication or a weak credential change management system. Consider the impact and potential mitigations for your custom authentication scheme.

Justification: Ensure 2FA before credential changes, Enforce strong password policies, Disallow blank passwords and prompt users to set a secure one, Use strong hashing algorithms for storing credentials.

Interaction: Web Response (HTTPS)



39. Web Application Process Memory Tampered [State: Mitigation Implemented] [Priority: Low]

Category: Tampering

Description: If Web Application is given access to memory, such as shared memory or pointers, or is given the ability to control what Browser Client executes (for example, passing back a function pointer.), then Web Application can tamper with Browser Client. Consider if the function could work with less access to memory, such as passing data rather than pointers. Copy in data provided, and then validate it.

Justification: Restrict access to memory areas, sanitize the input data and use a sandbox

40. Elevation Using Impersonation [State: Mitigation Implemented] [Priority: Low]

Category: Elevation Of Privilege

Description: Browser Client may be able to impersonate the context of Web Application in order to gain additional privilege.

Justification: We implemented strong authorization and authentication

41. Potential Data Repudiation by Browser Client [State: Mitigation Implemented] [Priority: Low]

Category: Repudiation

Description: Browser Client claims that it did not receive data from a source outside the trust boundary. Consider using logging or auditing to record the source, time, and summary of the received data.

Justification: Log and Audit Data Transfers. Implement strong authentication and authorization mechanisms.

42. Potential Process Crash or Stop for Browser Client [State: Mitigation Implemented] [Priority: Low]

Category: Denial Of Service

Description: Browser Client crashes, halts, stops or runs slowly; in all cases violating an availability metric.

Justification: We implemented a back up system to create room for redundancy. Cloudfare can also be implemented for ddos protection

43. Data Flow Web Response (HTTPS) Is Potentially Interrupted [State: Mitigation Implemented] [Priority: Low]

Category: Denial Of Service

Description: An external agent interrupts data flowing across a trust boundary in either direction.

Justification: We implemented a back up system to create room for redundancy. Cloudfare can also be implemented for ddos protection

44. Browser Client May be Subject to Elevation of Privilege Using Remote Code Execution [State: Mitigation Implemented] [Priority: Low]

Category: Elevation Of Privilege

Description: Web Application may be able to remotely execute code for Browser Client.

Justification: Escape and sanitize user-generated content tp prevent cross site scripting, Use CSRF tokens for form submissions.

45. Elevation by Changing the Execution Flow in Browser Client [State: Mitigation Implemented] [Priority: Low]

Category: Elevation Of Privilege

Description: An attacker may pass data into Browser Client in order to change the flow of program execution within Browser Client to the attacker's choosing.

Justification: Escape and sanitize user-generated content tp prevent cross site scripting, Use CSRF tokens for form submissions.

46. Weak Authentication Scheme [State: Mitigation Implemented] [Priority: Low]

Category: Information Disclosure

Description: Custom authentication schemes are susceptible to common weaknesses such as weak credential change management, credential equivalence, easily guessable credentials, null credentials, downgrade authentication or a weak credential change management system. Consider the impact and potential mitigations for your custom authentication scheme.

Justification: Ensure 2FA before credential changes, Enforce strong password policies, Disallow blank passwords and prompt users to set a secure one, Use strong hashing algorithms for storing credentials.

47. Replay Attacks [State: Mitigation Implemented] [Priority: Low]

Category: Tampering

Description: Packets or messages without sequence numbers or timestamps can be captured and replayed in a wide variety of ways. Implement or utilize an existing communication protocol that supports anti-replay techniques (investigate sequence numbers before timers) and strong integrity.

Justification: Include a timestamp in each message and reject messages that are too old. Sign messages using HMAC (Hash-based Message Authentication Code) with a shared secret key.

48. Collision Attacks [State: Mitigation Implemented] [Priority: Low]

Category: Tampering

Description: Attackers who can send a series of packets or messages may be able to overlap data. For example, packet 1 may be 100 bytes starting at offset 0. Packet 2 may be 100 bytes starting at offset 25. Packet 2 will overwrite 75 bytes of packet 1. Ensure you reassemble data before filtering it, and ensure you explicitly handle these sorts of cases.

Justification: Drop overlapping fragments, Reject packets that overwrite previous data. Detect and block malicious packets. Prevent unnecessary fragmentation. Validate message integrity.